Sklearn Library

BSc. Nhat-Tan Bui

Software Engineering Lab (SELab),
University of Science - VNUHCM
Artificial Intelligence and Computer Vision Lab (AICV),
University of Arkansas, Fayetteville
https://tanbuinhat.github.io/

Content

- Introduction
  - Loading dataset
  - Splitting dataset
- Linear Regression
- Classification
  - Logistic Regression
  - Support Vector Machines
  - K-nearest neighbors
  - Decision Trees
  - Random Forests
- Confusion Matrix

Scikit-learn (Sklearn)

- https://scikit-learn.org/stable/
- Open source for Machine Learning with Python
  - Simple and efficient tools for predictive data analysis
  - Built on NumPy, SciPy, and matplotlib

- Installing
  - pip install -U scikit-learn

Introduction

- Example Datasets

`load_iris`(*[, return_X_y, as_frame])	Load and return the iris dataset (classification).
`load_diabetes`(*[, return_X_y, as_frame, scaled])	Load and return the diabetes dataset (regression).
`load_digits`(*[, n_class, return_X_y, as_frame])	Load and return the digits dataset (classification).
`load_linnerud`(*[, return_X_y, as_frame])	Load and return the physical exercise Linnerud dataset.
`load_wine`(*[, return_X_y, as_frame])	Load and return the wine dataset (classification).
`load_breast_cancer`(*[, return_X_y, as_frame])	Load and return the breast cancer wisconsin dataset (classification).

  - X is Data for training
  - y is Label in case of supervised learning
  - return X_y is whether return tuple of data and label
  - as_frame is whether return data in Pandas DataFrame

Introduction

- Loading example dataset

```
from sklearn.datasets import load_digits

digit = load_digits()
X, y = digit['data'], digit['target']        # type: np.ndarray
```

```
from sklearn.datasets import load_digits

digit = load_digits()
X, y = digit['data'], digit['target']

X, y
```

```
(array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
        [ 0.,  0.,  0., ..., 10.,  0.,  0.],
        [ 0.,  0.,  0., ..., 16.,  9.,  0.],
        ...,
        [ 0.,  0.,  1., ...,  6.,  0.,  0.],
        [ 0.,  0.,  2., ..., 12.,  0.,  0.],
        [ 0.,  0., 10., ..., 12.,  1.,  0.]]), array([0, 1, 2, ..., 8, 9, 8]))
```

Introduction

- Splitting dataset intro training and testing subset (here)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
#70% for training and 30% for testing
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

print("X_train shape: ", X_train.shape, "Y_train shape: ", y_train.shape)
print("X_test shape: ", X_test.shape, "Y_test shape: ", y_test.shape)
```

```
X_train shape:  (1257, 64) Y_train shape:  (1257,)
X_test shape:  (540, 64) Y_test shape:  (540,)
```

Linear Regression (here)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

regr = LinearRegression() # define estimator and optional parameter
regr.fit(X_train, y_train) # fit model in training data

y_pred = regr.predict(X_test) # predict model in testing data
estimator = mean_squared_error(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy", regr.score(X_test, y_test)) # correct predictions / total
print("Estimated Regression Coefficients: ", regr.coef_) # coefficient need to find
print("Unbiased estimator: ", estimator) # value of Mean Squared Error
```

Classification

- Logistic Regression (here)

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error

regr = LogisticRegression() # define estimator and optional parameter
regr.fit(X_train, y_train) # fit model in training data

y_pred = regr.predict(X_test) # predict model in testing data
accuracy = accuracy_score(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy: ", accuracy) # print the accuracy
print("Prediction label: ", y_pred) # print prediction label for test set
```

Classification

- Support Vector Machines (here)

  SVC implements the “one-versus-one” while LinearSVC implements “one-vs-the-rest” multi-class approach for multi-class classification

```
from sklearn import svm
from sklearn.metrics import accuracy_score

svm = svm.SVC()# define estimator and optional parameter
svm.fit(X_train, y_train) # fit model in training data

y_pred = svm.predict(X_test) # predict model in testing data
accuracy = accuracy_score(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy: ", accuracy) # print the accuracy
print("Prediction label: ", y_pred) # print prediction label for test set
```

Classification

- Nearest Neighbors (here)

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors = 5) # define estimator and parameter
knn.fit(X_train, y_train) # fit model in training data

y_pred = knn.predict(X_test) # predict model in testing data
accuracy = accuracy_score(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy: ", accuracy) # print the accuracy
print("Prediction label: ", y_pred) # print prediction label for test set
```

Classification

- Decision Trees (here)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

dt = DecisionTreeClassifier() # define estimator and optional parameter
dt.fit(X_train, y_train) # fit model in training data

y_pred = dt.predict(X_test) # predict model in testing data
accuracy = accuracy_score(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy: ", accuracy) # print the accuracy
print("Prediction label: ", y_pred) # print prediction label for test set
```

Classification

- Random Forests (here)

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

rf = RandomForestClassifier () # define estimator and optional parameter
rf.fit(X_train, y_train) # fit model in training data

y_pred = rf.predict(X_test) # predict model in testing data
accuracy = accuracy_score(y_test, y_pred) # evaluate the accuracy of model

print("Accuracy: ", accuracy) # print the accuracy
print("Prediction label: ", y_pred) # print prediction label for test set
```

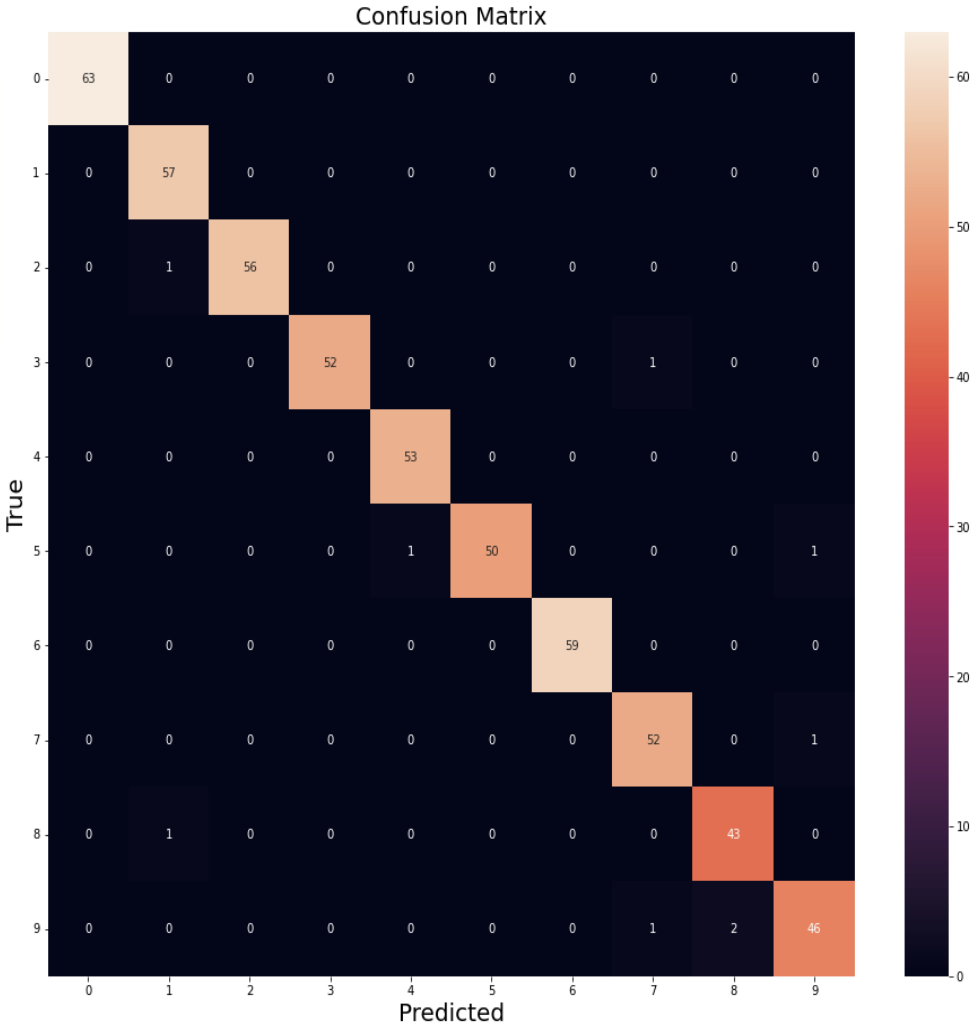
Confusion Matrix (here)

```
from sklearn import metrics
import seaborn as sns
import matplotlib.pyplot as plt

cm = metrics.confusion_matrix(y_test, y_pred)
class_names = list(digit.target_names)

fig = plt.figure(figsize=(16, 14))
ax= plt.subplot()
sns.heatmap(cm, annot=True, ax = ax, fmt = 'g')

ax.set_xlabel('Predicted', fontsize=20)
ax.xaxis.set_ticklabels(class_names, fontsize = 10)
ax.xaxis.tick_bottom()

ax.set_ylabel('True', fontsize=20)
ax.yaxis.set_ticklabels(class_names, fontsize = 10)
```

```
plt.yticks(rotation=0)

plt.title('Confusion Matrix', fontsize=20)
plt.show()
```